

JOB SKILL EXTRACTION USING NLP

Final Internship Project Report

Submitted by: Satyam Singh

Data Science and Analysis Intern

DECLARATION

I hereby declare that the project entitled “**NLP-Based Job Skills Extraction and Classification System**” has been completed by me as part of my internship at **TechnoSignia Software Institute**. The project focuses on applying **Natural Language Processing (NLP)** techniques to automatically extract relevant skills from job descriptions, normalize the extracted skills, and classify them into predefined categories. The application was developed using **Python, spaCy, FastAPI, Uvicorn, Pandas, and NLP techniques**.

I confirm that the work presented in this report is based on my learning, implementation, and practical work carried out during the internship. The project has been completed under the guidance and supervision of the assigned mentor at TechnoSignia.

Name: SATYAM SINGH

Internship Program: Data Analyst / NLP Project

Organization: TechnoSignia Software Institute

Date: _____

CERTIFICATE

CERTIFICATE OF COMPLETION

This is to certify that Mr. SATYAM SINGH has successfully completed the internship/project entitled "NLP-Based Job Skills Extraction and Classification System" at TechnoSignia Software Institute.

During the internship, the candidate worked on an NLP-based project focused on extracting skills from job descriptions and converting unstructured textual information into structured skill data. The project involved text preprocessing, skill extraction using spaCy and NLP techniques, skill normalization, skill categorization, model evaluation, error analysis, and development of a REST API using FastAPI and Uvicorn.

The candidate demonstrated dedication and successfully applied the concepts learned during the internship to the development of the project.

We wish them success in their future academic and professional endeavors.

Project Mentor: ___SANKET KANSE___

Designation: _____MENTOR_____

Organization: TechnoSignia Software Institute

Date: _____07/09/2026_____

Signature & Seal: _____

ABSTRACT

The NLP-Based Job Skills Extraction and Classification System is an NLP project developed during the internship at TechnoSignia Software Institute. The primary objective of the project is to automatically identify relevant skills from unstructured job descriptions and organize them into meaningful categories.

Job descriptions contain valuable information about the technical and professional skills required for different roles. However, manually identifying and analyzing these skills can be time-consuming. The proposed system addresses this problem by using Natural Language Processing techniques to process job descriptions and extract relevant skills.

The project involves multiple stages, including text preprocessing, skill extraction, skill normalization, skill categorization, model evaluation, and error analysis. spaCy is used for NLP processing and entity/skill extraction, while predefined skill taxonomies and matching techniques are used to normalize different skill names, synonyms, and variations. The extracted skills are mapped into categories such as Programming, Database, Cloud, Machine Learning, Data Analytics, and Business Intelligence.

To make the system usable as an application, a REST API was developed using FastAPI, with Uvicorn serving as the ASGI server. The API accepts a job description as input and returns the detected, normalized, and categorized skills in a structured format.

The project demonstrates the practical application of NLP in job-market analysis, recruitment support, skill-demand analysis, and automated job description processing, while providing hands-on experience in developing and deploying an NLP-based application.

Keywords: Natural Language Processing, Skill Extraction, Job Description, Skill Classification, Skill Normalization, spaCy, FastAPI, Uvicorn, REST API.

TABLE OF CONTENTS

1. Introduction	8
2. Problem Statement & Objectives	8
Problem Statement	8
Business Objective	8
Technical Objective	8
Expected Outcome	8
3. Project Scope & Use Case	9
Recruitment Use Case	9
Practical Applications	9
4. Technology & Tools Used	9
5. System Architecture & Methodology	9
6. Dataset Collection & Understanding	10
Dataset Fields	10
Data Inspection Findings	10
7. Exploratory Data Analysis (EDA)	11
Job Titles and Roles	11
Description Length	12
.....	12
Postings by Year	13
.....	13
.....	13
Missing Values and Duplicates	14
Most Frequent Words (post-cleaning)	15
8. Data Cleaning & NLP Preprocessing	15
9. Skill Taxonomy Development	16
Structure	16
10. Rule Based Skill Extraction	17
Dictionary / Regex Matching	17
spaCy PhraseMatcher	17
11. Statistical & Semantic NLP Techniques	18
TF-IDF and N-Grams	18
.....	19

Semantic Similarity (Sentence-BERT)	19
12. Named Entity Recognition (NER)	20
Custom NER.....	20
Standard (Pretrained) NER.....	20
13. Machine Learning Model	21
Feature Engineering	21
Initial Result and Diagnosis	22
Fix and Verification	23
Classification Pipeline	24
14. Transformer-Based Model	25
Pretrained Model — dslim/distilbert-NER.....	25
Fine-Tuning via Weak Supervision	25
Outcome	25
15. Model Evaluation & Comparison	26
16. Error Analysis	28
Remaining Error Patterns	28
.....	28
.....	29
17. Skill Normalization & Categorization.....	29
18. Dashboard & Data Visualization	30
KPI Cards	30
Charts	30
19. Application / API Development	31
Architecture	31
Application Interface	31
Input / Output Example	31
Authentication	31
API Endpoints	31
20. Testing & Results.....	32
Sample Test Cases	32
.....	32
21. Challenges & Limitations	33
Problems Faced	33
Model Limitations	33
Dataset Limitations	33

22. Conclusion	34
23. Future Scope.....	35
24. Internship Learning Outcomes	35
Technical Skills	35
Analytical Skills	35
Project-Development Skills	35
25. References.....	36
26. Appendix	37
A. Sample API Request	37
B. Sample API Response	37
C. Core Extraction Function (simplified)	37
D. Deliverable File Structure	37
E. Screenshots	38

1. Introduction

Job postings describe the skills a role requires in free-flowing, unstructured paragraphs rather than structured fields. This makes it difficult for recruiters, job seekers, and workforce analysts to search, filter, count, or trend skill demand — the information is there, but it isn't usable at scale without manual reading.

Natural Language Processing (NLP) is the branch of artificial intelligence concerned with enabling computers to read, interpret, and extract meaning from human language. It ranges from simple rule-based text matching to deep learning models (transformers) that learn contextual language patterns from large text corpora.

This project applies a spectrum of NLP techniques — from dictionary-based matching through to fine-tuned transformer models — to the specific task of job-skill extraction: automatically identifying which skills a job description is asking for, and grouping those skills into meaningful categories (Programming, Database, Business Intelligence, and so on). The project does not stop at a single technique; instead, four different extraction approaches were built, evaluated honestly against a hand-labeled sample, and compared head-to-head, with the best-performing combination shipped as a working web application backed by a REST API.

2. Problem Statement & Objectives

Problem Statement

Manually reading thousands of job postings to identify and tag required skills does not scale, and produces inconsistent results between reviewers. There is no automated system in place to convert a raw job description into a structured, categorized list of the skills it requires.

Business Objective

Build a system that lets three groups act on job posting data more effectively: recruiters can auto-tag postings for search and filtering; job seekers can compare their own resume's skills against a target posting; and workforce analysts can aggregate extracted skills across many postings to observe skill-demand trends.

Technical Objective

- Build and compare multiple skill-extraction methods: rule-based (dictionary/regex and spaCy), statistical/semantic, a trained ML classifier, and a fine-tuned transformer.
- Evaluate every method honestly against a hand-labeled ground-truth sample using Precision, Recall, and F1.
- Package the best-performing method as a reusable API and a working web application, with a Power BI analytics dashboard on top.

Expected Outcome

A working end-to-end system: a cleaned and explored dataset, a documented skill taxonomy, four evaluated extraction methods with an honest comparison, a login-protected web application exposing the winning method through an API, and a Power BI dashboard summarizing skill demand across the dataset.

3. Project Scope & Use Case

The project scope covers the full pipeline from raw job posting data to a deployed, usable application — it does not stop at a notebook-only analysis.

Recruitment Use Case

A recruiter pastes a job description into the application and instantly receives a structured, categorized list of the skills that posting requires — turning free text into filterable, searchable data without manual review.

Practical Applications

- Recruiters: auto-tagging postings with structured skills for search and filtering.
- Job seekers: comparing a resume's extracted skills against a target posting to identify gaps.
- Workforce analysts: aggregating extracted skills across thousands of postings to observe which skills are rising or declining in demand over time.

4. Technology & Tools Used

Category	Tools / Libraries
Language	Python 3.9+
Data Handling	Pandas, NumPy
NLP	spaCy (PhraseMatcher, custom NER, tokenization), NLTK-style preprocessing
Statistical / Semantic NLP	Scikit-learn (TF-IDF, Logistic Regression), Sentence-BERT (semantic similarity)
Deep Learning	Hugging Face Transformers (DistilBERT — pretrained and fine-tuned NER)
Model Evaluation	Scikit-learn metrics (Precision, Recall, F1, Confusion Matrix), sequeval
Database	SQLite (user accounts and sessions)
Application / API	FastAPI, Uvicorn, Pydantic
Frontend	HTML, CSS, vanilla JavaScript
Analytics Dashboard	Power BI (DAX measures, KPI cards, charts)
Environment	Google Colab / Jupyter Notebook, VS Code

5. System Architecture & Methodology

The project follows a linear pipeline from raw data to a deployed application, with an evaluation and error-analysis stage inserted before any method is chosen for production:

- Dataset Collection & Understanding
- Exploratory Data Analysis (EDA)
- Data Cleaning & NLP Preprocessing
- Skill Taxonomy Development

- Rule-Based Extraction (Dictionary/Regex + spaCy PhraseMatcher)
- Statistical & Semantic NLP (TF-IDF, N-Grams, Sentence-BERT)
- Custom Named Entity Recognition
- Machine Learning Classification (TF-IDF + Logistic Regression)
- Transformer-Based Extraction (pretrained, then fine-tuned via weak supervision)
- Model Evaluation & Comparison (Precision / Recall / F1)
- Error Analysis and root-cause fixes
- Skill Normalization & Category Mapping
- Dashboard (Power BI) and Application (FastAPI + web interface) deployment

The application-level architecture (Day 19) follows this flow for every user request:

User → Web Interface → Authentication → Preprocessing → Skill Extraction Model → Skill Normalization → Category Mapping → Final Skills

6. Dataset Collection & Understanding

The dataset consists of 5,000 real job postings with 23 fields, spanning postings from 2021 to 2023.

Dataset Fields

Group	Fields
Role	job_title, role, job_description, responsibilities
Requirements	qualifications, experience, skills
Logistics	work_type, salary_range, location, country
Company	company, company_size, company_profile, benefits
Timing	job_posting_date
Other	job_id, latitude, longitude, preference, contact_person, contact, job_portal

Data Inspection Findings

- 5,000 rows, 23 columns, no critical missing-value issues in the core fields used for extraction.
- 66+ unique job titles, but descriptions are heavily templated — the same wording repeats across many postings for a given role (e.g. "UX/UI Designer" appears 154 times, largely with identical description text).
- This templating was noted during initial inspection and later turned out to be the root cause of a serious modeling bug (see Section 21, Challenges & Limitations).

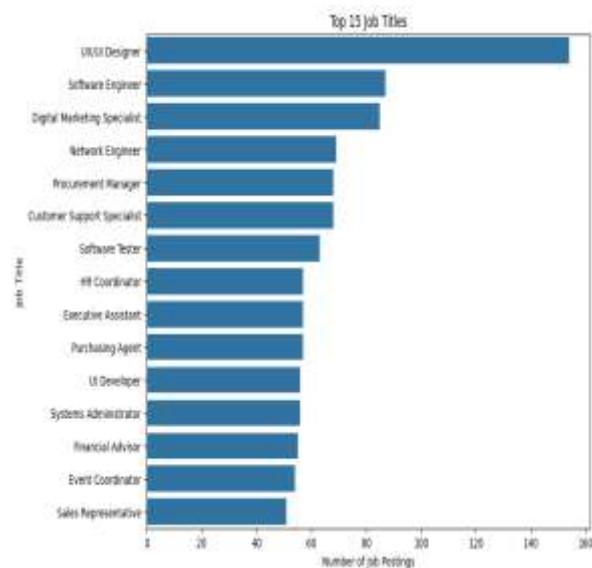
7. Exploratory Data Analysis (EDA)

Job Titles and Roles

The top job titles by posting count: UX/UI Designer (154), Software Engineer (87), Digital Marketing Specialist (85), Network Engineer (69), Procurement Manager (68), and Customer Support Specialist (68).

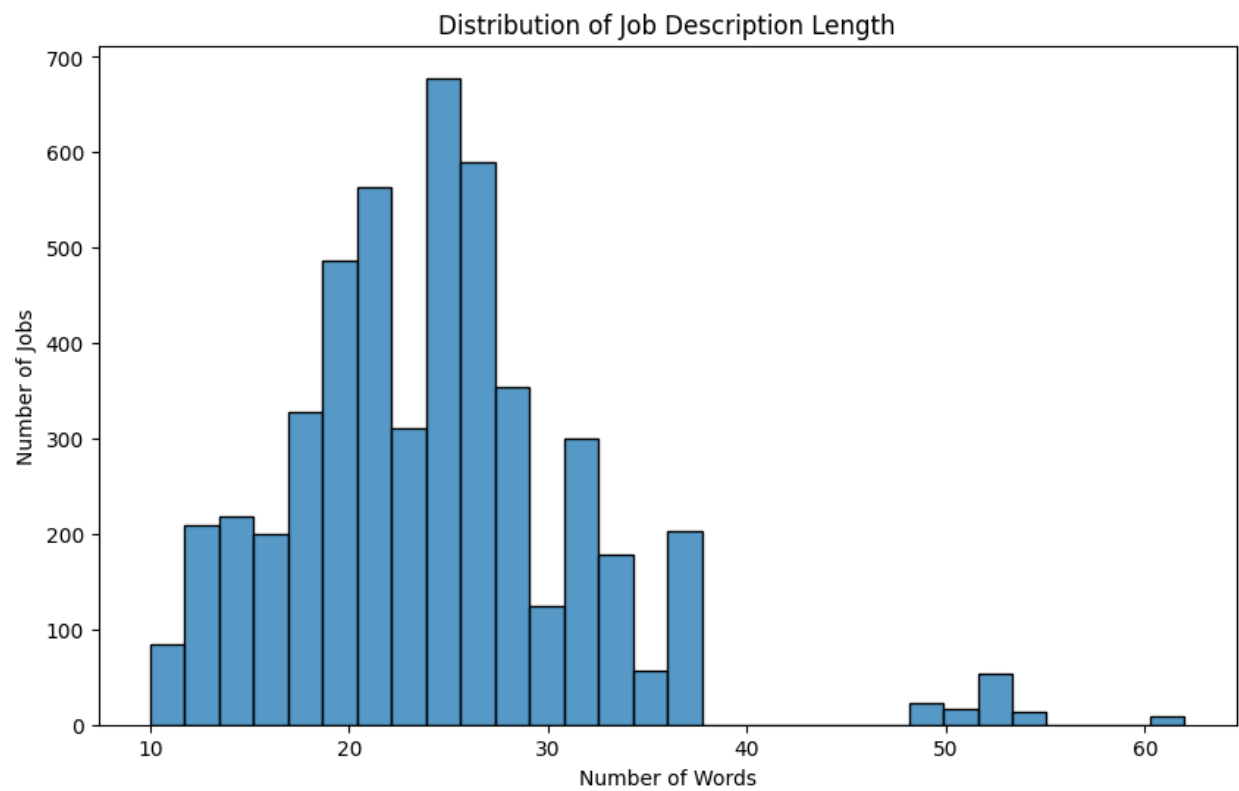
job_title	count
UX/UI Designer	154
Software Engineer	87
Digital Marketing Specialist	85
Network Engineer	69
Procurement Manager	68
Customer Support Specialist	68
Software Tester	63
HR Coordinator	57
Executive Assistant	57
Purchasing Agent	57
UI Developer	56
Systems Administrator	56
Financial Advisor	55
Event Coordinator	54
Sales Representative	51

dtype: int64



Description Length

Metric	Average
Words per posting	24.3
Characters per posting	186
Sentences per posting	1.6

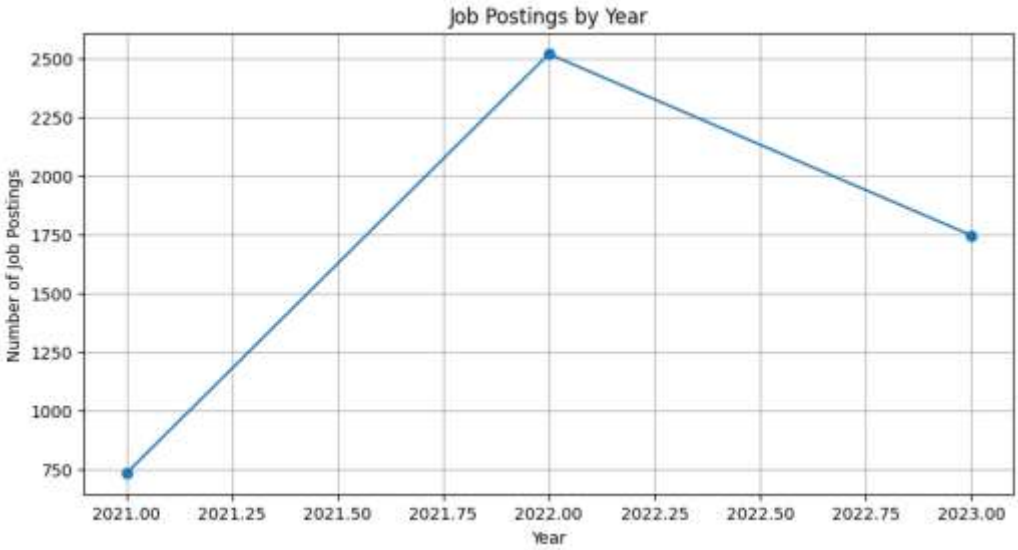


Postings by Year

Year	Postings
2021	732
2022	2,521
2023	1,747

	count
posting_year	
2021	732
2022	2521
2023	1747

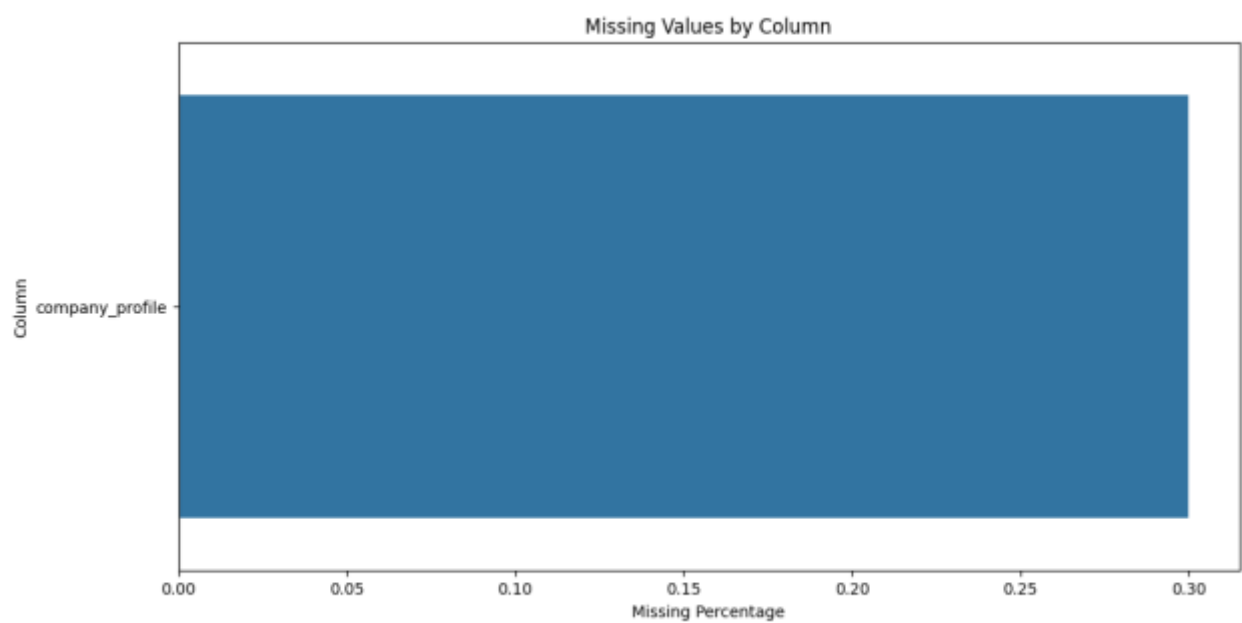
dtype: int64



Missing Values and Duplicates

Core fields (job_title, job_description, role) had no missing values. The main data-quality finding was not missing data but duplication: many postings share identical or near-identical description text, which was flagged for careful handling during model training and evaluation.

	Missing Count	Missing Percentage
Company Profile	15	0.3
Experience	0	0.0
Job Id	0	0.0
Salary Range	0	0.0
location	0	0.0
Country	0	0.0
Qualifications	0	0.0
latitude	0	0.0
longitude	0	0.0
Company Size	0	0.0
Work Type	0	0.0
Preference	0	0.0
Contact Person	0	0.0
Contact	0	0.0
Job Posting Date	0	0.0
Job Title	0	0.0
Role	0	0.0
Job Description	0	0.0
Job Portal	0	0.0
Benefits	0	0.0
skills	0	0.0
Responsibilities	0	0.0
Company	0	0.0



Most Frequent Words (post-cleaning)

data (982), user (806), support (743), design (674), manage (608), develop (604), financial (586).



```
***
      word  frequency
0    ensure      998
1     data      982
2     user      806
3   support      743
4   provide      712
5    create      695
6    design      674
7  ensuring      626
8   manage      608
9   develop      604
10  financial      586
11    work      547
12  analyze      512
13 marketing      481
```

	word	frequency
0	ensure	998
1	data	982
2	user	806
3	support	743
4	provide	712
5	create	695
6	design	674
7	ensuring	626
8	manage	608
9	develop	604
10	financial	586
11	work	547
12	analyze	512
13	marketing	481

8. Data Cleaning & NLP Preprocessing

- Missing & duplicate check: every column audited for nulls; job_id checked for duplicate postings.
- Schema standardization: column names normalized to lowercase snake_case.
- Date parsing: job_posting_date parsed to datetime, with posting_year extracted for trend analysis.
- Text normalization: descriptions lowercased, punctuation stripped.
- Tokenization: descriptions split into word-level tokens using spaCy's tokenizer.
- Stopword removal: common non-informative words ("the", "and", "with", etc.) removed before frequency analysis.
- Lemmatization: word forms reduced to their base/dictionary form where applicable, aiding downstream matching consistency.

- Sentence segmentation: descriptions split into sentence units using spaCy's sentence boundary detection, supporting the average-sentence-count EDA metric.

```
tokens:
['am', 'are', 'looking', 'for', 'candidates', 'with', 'strong', 'experience', 'in', 'python', 'and', 'sql', '.', 'the', 'candidate', 'should', 'have', 'good',
```

```
• After Stopword Removal:
['looking', 'candidates', 'strong', 'experience', 'python', 'sql', 'candidate', 'good', 'knowledge', 'power', 'bi', 'aws']
```

9. Skill Taxonomy Development

A custom skill taxonomy was built by hand to serve as the ground vocabulary for every rule-based and evaluation step in the project.

Metric	Count
Categories	37
Canonical skills	368
Synonyms indexed	1,166

Structure

The taxonomy is organized as: category → canonical skill → list of synonyms/aliases. For example, the canonical skill "Power BI" (category: Business Intelligence) is matched by synonyms including "power bi" and other common phrasings encountered in the dataset. This structure is what allows raw text mentions to be normalized into one consistent, categorized name regardless of how the skill was phrased in the original posting.

```
# JOB SKILL TAXONOMY - SYNONYMS / ALIASES
# =====
skill_taxonomy = {

    # =====
    # 1. PROGRAMMING
    # =====
    "Programming": {

        "Python": [
            "python",
            "python programming",
            "python development",
            "python scripting",
            "python language"
        ],

        "Java": [
            "java",
            "java programming",
            "java development",
            "java language"
        ],

        "C": [
            "c",
            "c programming",
            "c language"
        ],

        "C++": [
            "c++",
            "cpp",
            "c plus plus",
            "c++ programming"
        ]
    }
}
```


10. Rule Based Skill Extraction

Two independent rule-based methods were implemented against the same taxonomy vocabulary:

Dictionary / Regex Matching

Every taxonomy synonym is compiled into a word-boundary regex pattern (`\bsynonym\b`) and searched against the lowercased description text. A match returns the synonym's canonical skill name and category.

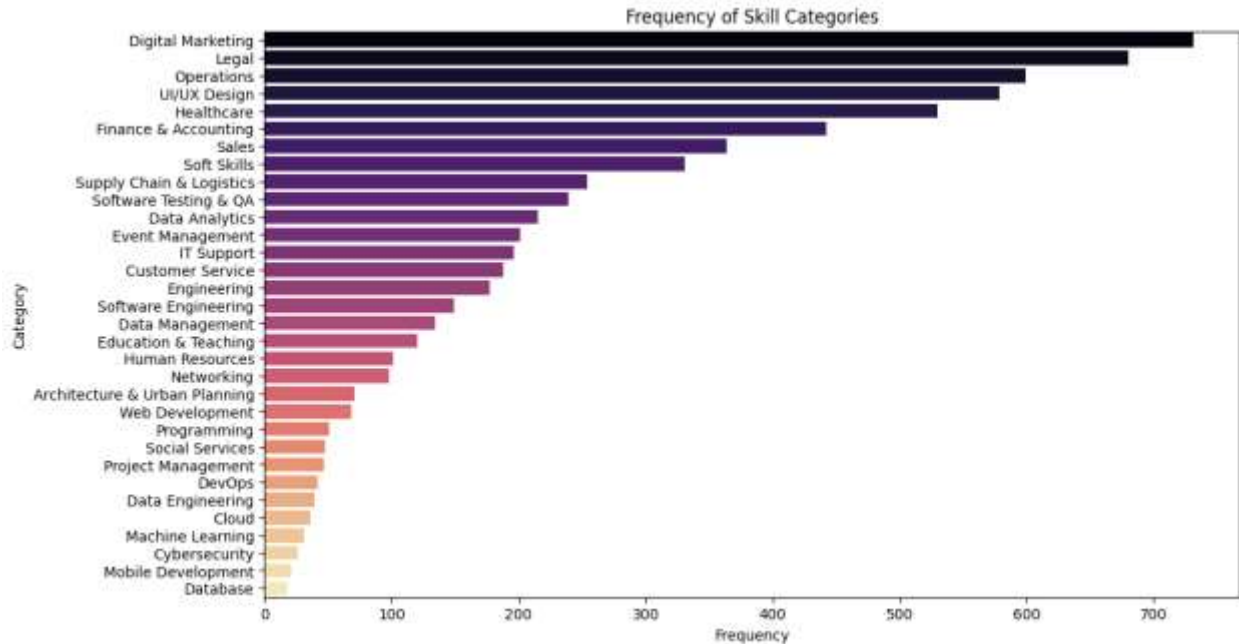
spaCy PhraseMatcher

A second, independent pass using spaCy's PhraseMatcher over the same synonym vocabulary, matched against the tokenized document rather than raw string search. This provides a cross-check against the regex method and, in evaluation, was found to catch some matches the regex method missed.

Strengths: millisecond inference, no GPU required, fully explainable (the exact synonym that matched is always known), and no training data required.

Limitations: both methods only catch wording already present in the taxonomy; a skill mentioned in phrasing that doesn't match any indexed synonym is missed entirely.

```
[{'category': 'Data Science',  
  'canonical_skill': 'Data Science',  
  'matched_synonym': 'data scientist'},  
{ 'category': 'Programming',  
  'canonical_skill': 'Python',  
  'matched_synonym': 'python'},  
{ 'category': 'Machine Learning',  
  'canonical_skill': 'Scikit-learn',  
  'matched_synonym': 'scikit-learn'},  
{ 'category': 'Machine Learning',  
  'canonical_skill': 'XGBoost',  
  'matched_synonym': 'xgboost'},  
{ 'category': 'Database', 'canonical_skill': 'SQL', 'matched_synonym': 'sql'},  
{ 'category': 'Database',  
  'canonical_skill': 'MySQL',  
  'matched_synonym': 'mysql'},  
{ 'category': 'Cloud', 'canonical_skill': 'AWS', 'matched_synonym': 'aws'}]
```

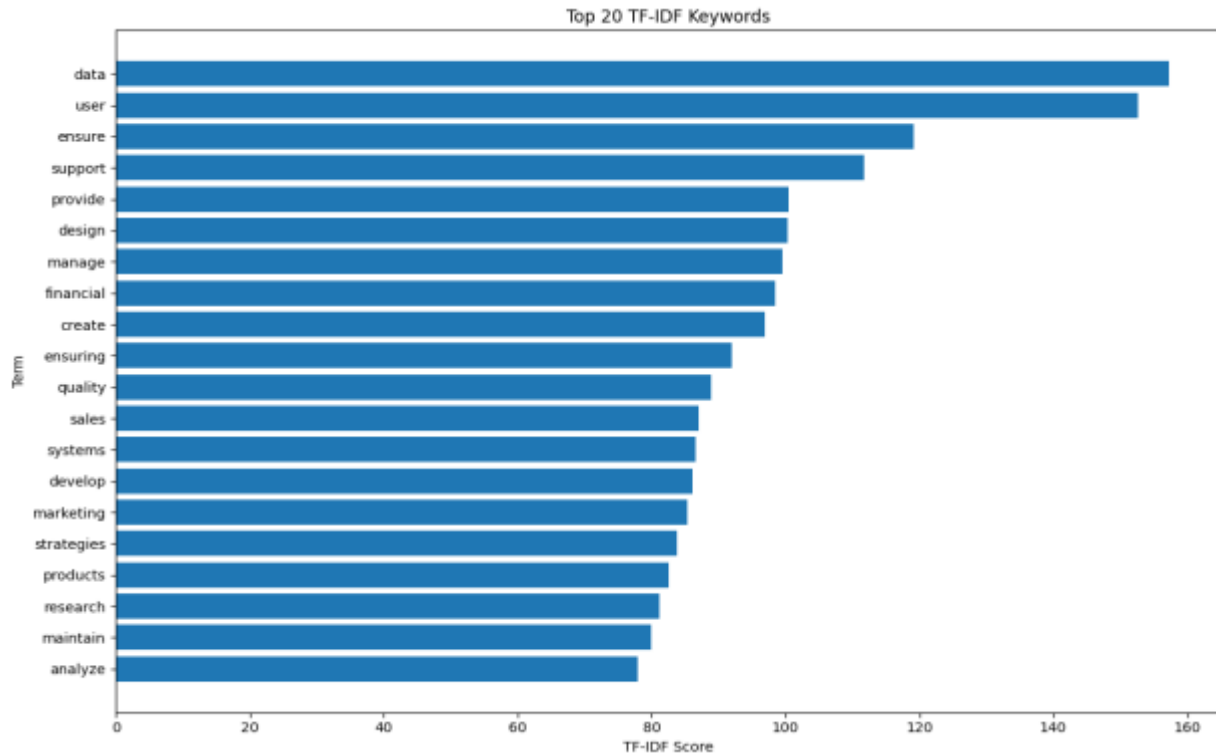


11. Statistical & Semantic NLP Techniques

TF-IDF and N-Grams

Term Frequency–Inverse Document Frequency (TF-IDF) was applied across the 5,000 descriptions, using unigrams through trigrams, to identify statistically important terms per description and to provide the feature representation later used by the Logistic Regression classifier (Section 13).

	term	tfidf_score
0	data	157.289586
1	user	152.696132
2	ensure	119.187561
3	support	111.761686
4	provide	100.406095
5	design	100.308497
6	manage	99.482207
7	financial	98.431219
8	create	96.950658
9	ensuring	92.029080
10	quality	88.870584



Semantic Similarity (Sentence-BERT)

To move beyond exact keyword matching, sentence embeddings were generated for both job descriptions and taxonomy skill phrases using a Sentence-BERT model, and cosine similarity was used to identify semantically related mentions — skills phrased differently from any exact taxonomy synonym, but semantically equivalent to a known skill.

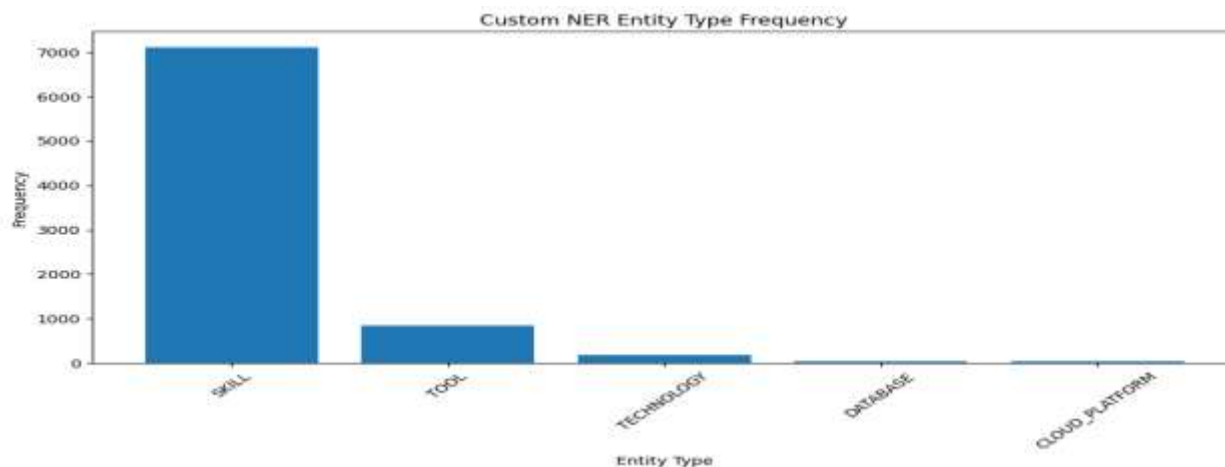
	ML	Machine Learning	NLP	Natural Language Processing	AI	Artificial Intelligence	PowerBI	Power BI
ML	1.000	0.373	0.181	0.097	0.344	0.299	0.099	0.094
Machine Learning	0.373	1.000	0.416	0.482	0.546	0.703	0.234	0.198
NLP	0.181	0.416	1.000	0.602	0.402	0.413	0.183	0.123
Natural Language Processing	0.097	0.482	0.602	1.000	0.311	0.429	0.150	0.114
AI	0.344	0.546	0.402	0.311	1.000	0.791	0.231	0.204
Artificial Intelligence	0.299	0.703	0.413	0.429	0.791	1.000	0.205	0.169
PowerBI	0.099	0.234	0.183	0.150	0.231	0.205	1.000	0.682
Power BI	0.094	0.198	0.123	0.114	0.204	0.169	0.682	1.000

12. Named Entity Recognition (NER)

Custom NER

A custom spaCy NER component was trained to recognize entities relevant to job descriptions, applied across all postings and stored alongside a human-readable entity/label rendering for inspection.

```
('Strong skills in python are preferred.', {'entities': [(17, 23, 'TECHNOLOGY')]}))
('Experience with python programming is required.', {'entities': [(16, 34, 'TECHNOLOGY')]}))
('Candidates should have knowledge of python development.', {'entities': [(36, 54, 'TECHNOLOGY')]}))
('Experience working with java is desirable.', {'entities': [(24, 28, 'TECHNOLOGY')]}))
('Strong skills in java programming are preferred.', {'entities': [(17, 33, 'TECHNOLOGY')]}))
('Strong skills in java development are preferred.', {'entities': [(17, 33, 'TECHNOLOGY')]}))
('The role requires experience using c.', {'entities': [(26, 27, 'TECHNOLOGY')]}))
('Knowledge of c programming is an advantage.', {'entities': [(13, 26, 'TECHNOLOGY')]}))
('Experience working with c language is desirable.', {'entities': [(24, 34, 'TECHNOLOGY')]}))
('Candidates should have knowledge of c++.', {'entities': [(36, 39, 'TECHNOLOGY')]}))
('Strong skills in cpp are preferred.', {'entities': [(17, 20, 'TECHNOLOGY')]}))
('Knowledge of c plus plus is an advantage.', {'entities': [(13, 24, 'TECHNOLOGY')]}))
('Knowledge of c# is an advantage.', {'entities': [(13, 15, 'TECHNOLOGY')]}))
('Knowledge of c sharp is an advantage.', {'entities': [(13, 20, 'TECHNOLOGY')]}))
('Experience with csharp is required.', {'entities': [(16, 22, 'TECHNOLOGY')]}))
('The role requires experience using javascript.', {'entities': [(35, 45, 'TECHNOLOGY')]}))
('Experience working with java script is desirable.', {'entities': [(24, 35, 'TECHNOLOGY')]}))
('Strong skills in js are preferred.', {'entities': [(17, 19, 'TECHNOLOGY')]}))
('Experience working with typescript is desirable.', {'entities': [(24, 34, 'TECHNOLOGY')]}))
('Experience with type script is required.', {'entities': [(16, 27, 'TECHNOLOGY')]}))
```



Standard (Pretrained) NER

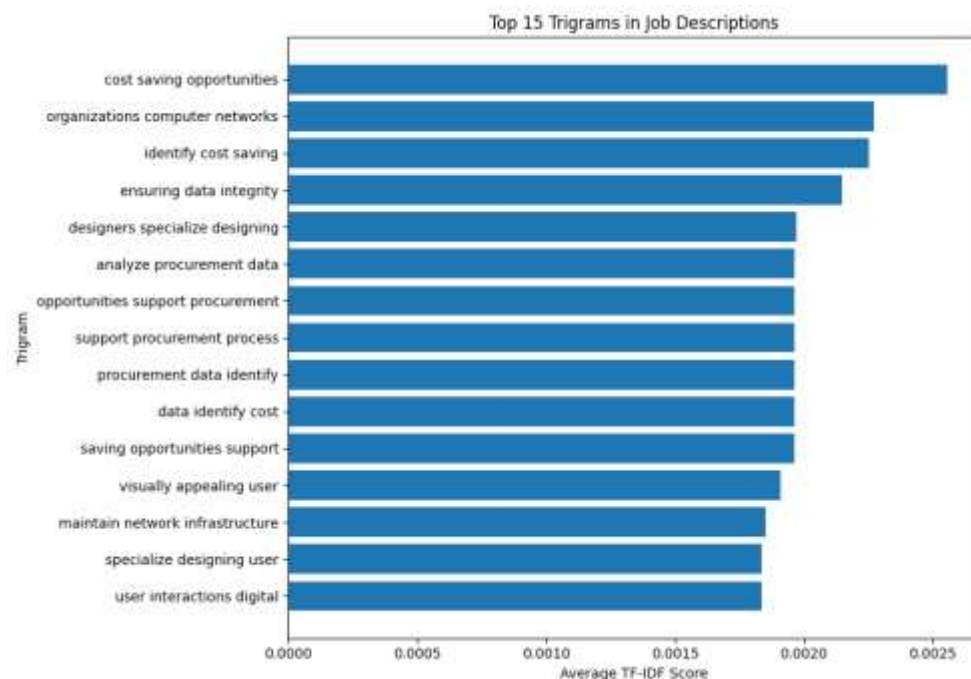
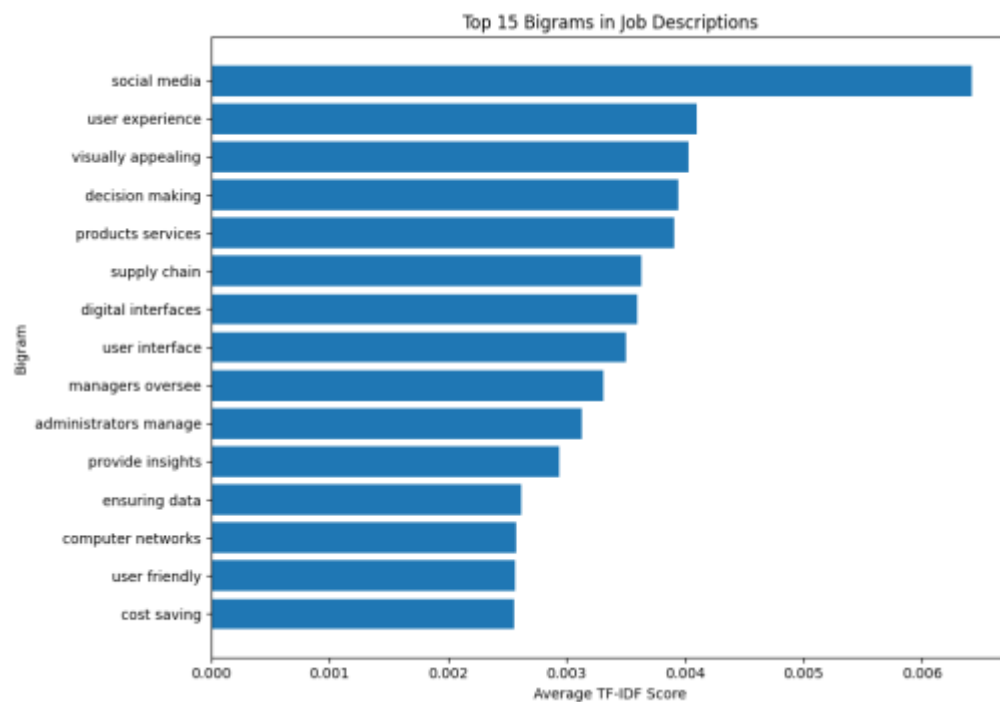
The pretrained dslim/distilbert-NER model was also tested. Its standard entity classes are PERSON, ORGANIZATION, LOCATION, and MISCELLANEOUS — there is no SKILL entity type in its training data. When tested against a technical, skill-dense job description, it extracted zero skill entities, since the concept of "skill" simply does not exist in its label space. This finding directly motivated the fine-tuning work described in Section 14.

13. Machine Learning Model

A TF-IDF + Logistic Regression classifier was trained to predict a job description's skill category.

Feature Engineering

TF-IDF vectors (unigrams to trigrams, English stopwords removed, sublinear term-frequency scaling) were computed over the `job_description_matching` text field.



Initial Result and Diagnosis

The initial train/test split (a plain random split) produced 100% test accuracy — an implausible result for a natural-language classification task. Investigation traced this to the dataset's heavy templating (Section 6): identical description text existed in both the training and test sets, so the model was memorizing exact text-to-label pairs rather than learning generalizable patterns. This was confirmed directly: when tested on a genuinely novel sentence — "Python developer... Django, REST API" — the model predicted "UI/UX Design" with only 25% confidence, showing no real understanding of the input.

```
print("Original records:", len(df))
print("ML records:", len(ml_df))
```

```
Original records: 5000
ML records: 3835
```

```
#Training Logistic Regression
classifier = LogisticRegression(
    max_iter=2000,
    random_state=42
)

classifier.fit(
    X_train_tfidf,
    y_train
)
```

LogisticRegression ⓘ ?

```
LogisticRegression(max_iter=2000, random_state=42)
```

```
#Accuracy
accuracy = accuracy_score(
    y_test,
    y_pred
)

print(
    f"Model Accuracy: {accuracy:.2%}"
)
```

```
Model Accuracy: 100.00%
```

Fix and Verification

The split was replaced with a GroupShuffleSplit keyed on the description text itself, guaranteeing that every duplicate of a given template lands entirely in train or entirely in test. A verification cell confirms zero description templates appear in both sets.

	precision	recall	f1-score	support
Architecture & Urban Planning	1.00	1.00	1.00	7
Cloud	1.00	1.00	1.00	2
Customer Service	1.00	1.00	1.00	20
Cybersecurity	1.00	1.00	1.00	5
Data Analytics	1.00	1.00	1.00	24
Data Engineering	1.00	1.00	1.00	8
Data Management	1.00	1.00	1.00	18
Database	1.00	1.00	1.00	4
DevOps	1.00	1.00	1.00	4
Digital Marketing	1.00	1.00	1.00	105
Education & Teaching	1.00	1.00	1.00	15
Engineering	1.00	1.00	1.00	24
Event Management	1.00	1.00	1.00	23
Finance & Accounting	1.00	1.00	1.00	30
Healthcare	1.00	1.00	1.00	55
Human Resources	1.00	1.00	1.00	14
IT Support	1.00	1.00	1.00	19
Legal	1.00	1.00	1.00	72
Machine Learning	1.00	1.00	1.00	2
Mobile Development	1.00	1.00	1.00	4
Networking	1.00	1.00	1.00	18
Operations	1.00	1.00	1.00	45
Programming	1.00	1.00	1.00	8
Project Management	1.00	1.00	1.00	6
Sales	1.00	1.00	1.00	50
Social Services	1.00	1.00	1.00	2
Soft Skills	1.00	1.00	1.00	32
Software Engineering	1.00	1.00	1.00	26
Software Testing & QA	1.00	1.00	1.00	31
Supply Chain & Logistics	1.00	1.00	1.00	20
UI/UX Design	1.00	1.00	1.00	65
Web Development	1.00	1.00	1.00	9
accuracy			1.00	767
macro avg	1.00	1.00	1.00	767
weighted avg	1.00	1.00	1.00	767

Classification Pipeline

TF-IDF vectorizer → Logistic Regression classifier → prediction, with an optional multi-label upgrade (predicting all skill categories present in a description, not only the dominant one) built as a stronger, more honest formulation of the task.

```
print("\nSaved File:")  
print("skill_classifier.pkl")
```



```
=====
DAY 13 – MODEL SUMMARY
=====

Dataset Size: 3835
Training Size: 3068
Testing Size: 767
Number of Categories: 32
Accuracy: 100.00%

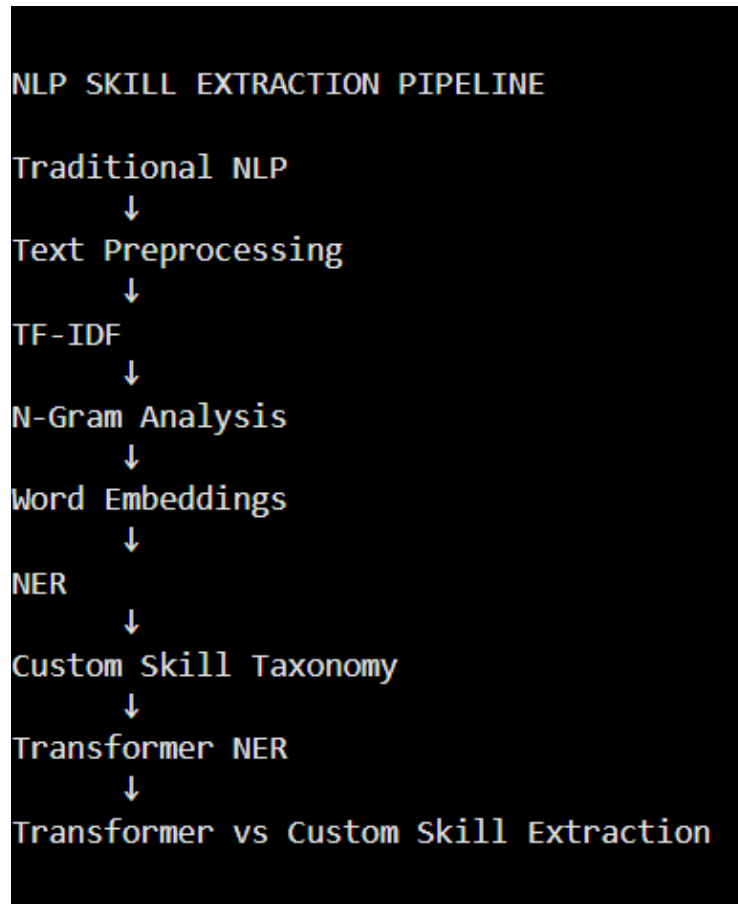
Model:
TF-IDF + Logistic Regression

Saved File:
skill_classifier.pkl
```


14. Transformer-Based Model

Pretrained Model — dslim/distilbert-NER

As established in Section 12, this pretrained DistilBERT-based NER model extracted zero skill entities from the evaluation sample, since its label space (PERSON/ORG/LOC/MISC) has no concept of "skill."



Fine-Tuning via Weak Supervision

Rather than manually annotating training data, the taxonomy's own regex matches were used to automatically generate BIO-format training labels (B-SKILL / I-SKILL / O) for every description with at least one taxonomy match. A fresh token-classification head was then fine-tuned on top of a DistilBERT base model using these auto-generated labels — no manual annotation was required.

Outcome

The fine-tuned model moved from 0 extracted skills (pretrained) to real extraction capability, correctly identifying skills such as Python, SQL, and AWS in test sentences that the pretrained model had completely missed. Its overall evaluation score, however, still trailed the two rule-based methods (see Section 15) — a realistic outcome for a first fine-tuning pass on weakly-labeled data, and an area flagged for further work (Section 23).

Day 14 shape: (100, 4)

Day 14 columns:

```
[ "job_title", "job_description_matching", "transformer_entities", "taxonomy_skills"]
```

	job_title	job_description_matching	transformer_entities	taxonomy_skills
0	Digital Marketing Specialist	social media managers oversee an organization's social media presence. they create and schedule content, engage with followers, and analyze social media metrics to drive brand awareness and engagement...		[[{"category": "Digital Marketing", "canonical_skill": "Social Media", "matched_synonym": "social media"}, {"category": "Digital Marketing", "canonical_skill": "Brand Marketing", "matched_synonym": "brand awareness"}]]
1	Web Developer	frontend web developers design and implement user interfaces for websites, ensuring they are visually appealing and user-friendly. they collaborate with designers and backend developers to create...		[[{"category": "Web Development", "canonical_skill": "Web Development", "matched_synonym": "web developers"}]]
2	Operations Manager	quality control managers establish and enforce quality standards within an organization. they develop quality control processes, perform inspections, and implement corrective actions to maintain...		[[{"category": "Software Testing & QA", "canonical_skill": "Quality Control", "matched_synonym": "quality control"}]]
3	Network Engineer	wireless network engineers design, implement, and maintain wireless network solutions. they optimize wireless connectivity, troubleshoot issues, and ensure reliable and secure wireless communication...		[[{"category": "Networking", "canonical_skill": "Wireless Networking", "matched_synonym": "wireless network"}, {"category": "Networking", "canonical_skill": "Wireless Networking", "matched_synonym": "wireless network"}]]
4	Event Manager	a conference manager coordinates and manages conferences, meetings, and events. they plan logistics, handle budgeting, liaise with vendors, and ensure the smooth execution of events, catering to...		[[{"category": "Finance & Accounting", "canonical_skill": "Budgeting", "matched_synonym": "budgeting"}, {"category": "Event Management", "canonical_skill": "Logistics", "matched_synonym": "logistics"}]]

	job_title	taxonomy_skills
0	Digital Marketing Specialist	[[{"category": "Digital Marketing", "canonical_skill": "Social Media", "matched_synonym": "social media"}, {"category": "Digital Marketing", "canonical_skill": "Brand Marketing", "matched_synonym": "brand awareness"}]]
1	Web Developer	[[{"category": "Web Development", "canonical_skill": "Web Development", "matched_synonym": "web developers"}]]
2	Operations Manager	[[{"category": "Software Testing & QA", "canonical_skill": "Quality Control", "matched_synonym": "quality control"}]]
3	Network Engineer	[[{"category": "Networking", "canonical_skill": "Wireless Networking", "matched_synonym": "wireless network"}, {"category": "Networking", "canonical_skill": "Wireless Networking", "matched_synonym": "wireless network"}]]
4	Event Manager	[[{"category": "Finance & Accounting", "canonical_skill": "Budgeting", "matched_synonym": "budgeting"}, {"category": "Event Management", "canonical_skill": "Logistics", "matched_synonym": "logistics"}]]
5	Software Tester	[[{"category": "Software Testing & QA", "canonical_skill": "Quality Assurance", "matched_synonym": "quality assurance"}]]
6	Teacher	[[{"category": "Education & Teaching", "canonical_skill": "Teaching", "matched_synonym": "teacher"}, {"category": "Education & Teaching", "canonical_skill": "Teaching", "matched_synonym": "instruct"}]]
7	UX/UI Designer	[[{"category": "UI/UX Design", "canonical_skill": "UI Design", "matched_synonym": "user interface designers"}, {"category": "UI/UX Design", "canonical_skill": "UI Design", "matched_synonym": "interact"}]]
8	UX/UI Designer	[[{"category": "UI/UX Design", "canonical_skill": "UI Design", "matched_synonym": "digital interfaces"}, {"category": "UI/UX Design", "canonical_skill": "UX Design", "matched_synonym": "user experience"}]]
9	Wedding Planner	[[{"category": "Event Management", "canonical_skill": "Budget Management", "matched_synonym": "budget management"}]]

15. Model Evaluation & Comparison

All four methods were evaluated against the same 100-row hand-labeled sample, scored using Precision, Recall, and F1 (set-overlap metrics), after aligning the hand-written ground truth to the taxonomy's own canonical skill vocabulary to avoid penalizing correct predictions for wording differences alone.

Method	Precision	Recall	F1
Taxonomy (Dictionary/Regex)	0.68	0.44	0.53
spaCy PhraseMatcher	0.66	0.47	0.55
Transformer (pretrained)	0.00	0.00	0.00
Transformer (fine-tuned)	0.34	0.27	0.30

...

	Method	Precision	Recall	F1	TP	FP	FN
0	Taxonomy (Dictionary/Regex)	0.6788	0.4408	0.5345	93	44	118
1	spaCy PhraseMatcher	0.6644	0.4692	0.5500	99	50	112
2	Transformer (pretrained)	0.0000	0.0000	0.0000	0	0	211
3	Transformer (fine-tuned)	0.3353	0.2701	0.2992	57	113	154

Saved: model_comparison.csv

The spaCy PhraseMatcher achieved the highest F1 score (0.55), narrowly ahead of the Taxonomy regex matcher (0.53); both dictionary-based methods clearly outperformed both transformer variants. For the Day 13 classifier, a confusion matrix (using the full 32-category label set, explicitly passed to both the `confusion_matrix` and `ConfusionMatrixDisplay` calls to keep the label axes aligned) was used to inspect category-level performance and confirm which categories had sufficient template diversity for a fair test-set evaluation.

Production decision: the Taxonomy + spaCy union (ensemble) was selected for deployment, since the two methods do not fully overlap — combining their predictions raises recall further than either method alone, while avoiding the transformer's higher compute cost for a currently lower score.

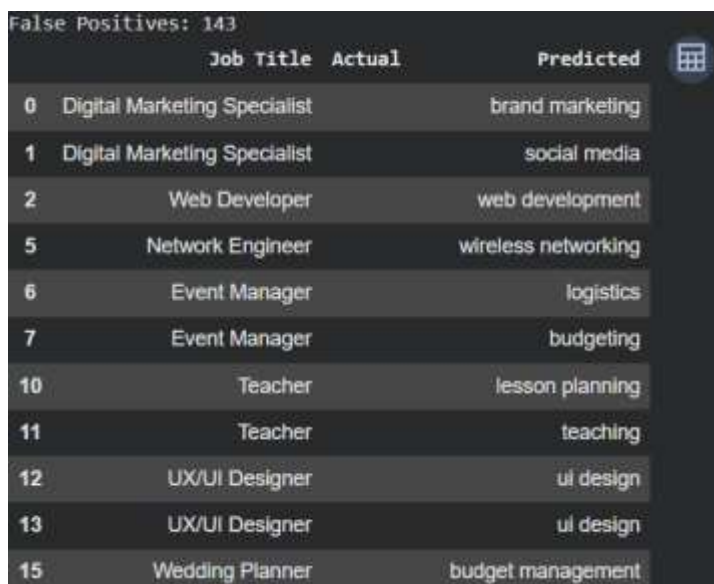
16. Error Analysis

Every major metric in this project was wrong on the first pass, for a specific and traceable reason. Diagnosing and fixing these was a core part of the project's actual work.

Symptom	Root Cause	Fix
Logistic Regression scored 100% test accuracy	Duplicate description templates split across train/test	GroupShuffleSplit keyed on description text
Transformer extracted 0 skills	Pretrained NER has no SKILL entity class	Fine-tuned via weak supervision (Section 14)
Evaluation accuracy collapsed to ~2%	Two different random samples merged incorrectly, leaving 64% of rows with no ground truth	Generate all predictions on one shared labeled sample
Recall stayed low even after the above fix	Hand-written ground-truth labels used different wording than the taxonomy's canonical names	Relabeled the evaluation set against the taxonomy's real vocabulary

Remaining Error Patterns

- False negatives: skills phrased in ways not covered by any indexed synonym (e.g. abbreviations, less common phrasings) are missed by both rule-based methods.
- False positives: the fine-tuned transformer produces more false positives (113) than either dictionary method despite finding fewer true positives (57), consistent with a model still early in training.
- Category coverage gaps: a small number of evaluation rows (purely administrative role descriptions) had no matching canonical skill in the taxonomy at all — a genuine taxonomy coverage gap rather than an extraction failure.



False Positives: 143

	Job Title	Actual	Predicted
0	Digital Marketing Specialist		brand marketing
1	Digital Marketing Specialist		social media
2	Web Developer		web development
5	Network Engineer		wireless networking
6	Event Manager		logistics
7	Event Manager		budgeting
10	Teacher		lesson planning
11	Teacher		teaching
12	UX/UI Designer		ui design
13	UX/UI Designer		ui design
15	Wedding Planner		budget management

P/FIN analysis created successfully.

	job_title	actual_skill_set	predicted_skill_set	× false_positive_skills	false_negative_skills
0	Digital Marketing Specialist	()	{brand marketing, social media}	{brand marketing, social media}	()
1	Web Developer	()	{web development}	{web development}	()
2	Operations Manager	{process development, inspection, quality control}	{quality control}	()	{process development, inspection}
3	Network Engineer	()	{wireless networking}	{wireless networking}	()
4	Event Manager	()	{logistics, budgeting}	{logistics, budgeting}	()
5	Software Tester	{testing, defect identification, quality assurance}	{quality assurance}	()	{testing, defect identification}
6	Teacher	()	{lesson planning, teaching}	{lesson planning, teaching}	()
7	UX/UI Designer	()	{ui design}	{ui design}	()
8	UX/UI Designer	{ux design, interaction design}	{ux design, ui design}	{ui design}	{interaction design}
9	Wedding Planner	()	{budget management}	{budget management}	()

17. Skill Normalization & Categorization

Raw matches from any extraction method are normalized to a single canonical_skill name via the taxonomy's synonym-to-skill mapping, and every canonical skill carries a fixed category. This normalization step is what allows different phrasings ("power bi", "microsoft power bi") to collapse into one consistent output ("Power BI", category: Business Intelligence), and is what makes the extracted skills usable for aggregation, deduplication, and the downstream Power BI dashboard.

In the production ensemble, results from the Taxonomy and spaCy methods are merged by canonical_skill name before being returned, ensuring no skill is duplicated in the final output even if both methods detected it independently.

Shape: (100, 3)
Columns: ['job_title', 'job_description_matching', 'normalized_skills']

	job_title	job_description_matching	normalized_skills
0	Digital Marketing Specialist	social media managers oversee an organizations social media presence. they create and schedule content, engage with followers, and analyze social media metrics to drive brand awareness and engagem...	['Social Media', 'Brand Marketing']
1	Web Developer	frontend web developers design and implement user interfaces for websites, ensuring they are visually appealing and user-friendly. they collaborate with designers and backend developers to create ...	['Web Development']
2	Operations Manager	quality control managers establish and enforce quality standards within an organization. they develop quality control processes, perform inspections, and implement corrective actions to maintain p...	['Quality Control']
3	Network Engineer	wireless network engineers design, implement, and maintain wireless network solutions. they optimize wireless connectivity, troubleshoot issues, and ensure reliable and secure wireless communicat...	['Wireless Networking']
4	Event Manager	a conference manager coordinates and manages conferences, meetings, and events. they plan logistics, handle budgeting, liaise with vendors, and ensure the smooth execution of events, catering to t...	['Budgeting', 'Logistics']
5	Software Tester	a quality assurance analyst tests software and products to ensure they meet quality standards. they identify defects, report issues, and work with development teams to resolve problems.	['Quality Assurance']
6	Teacher	a classroom teacher educates students in a specific subject or grade level. they create lesson plans, deliver instruction, assess student progress, and foster a positive learning environment.	['Teaching', 'Lesson Planning']
7	UX/UI Designer	user interface designers focus on the visual and interactive aspects of digital interfaces. they design layouts, buttons, and other elements to ensure a cohesive and visually appealing user inter...	['UI Design']
8	UX/UI Designer	interaction designers specialize in designing user interactions within digital interfaces. they create meaningful and engaging user experiences by considering user behaviors and system responses.	['UI Design', 'Ux Design']
9	Wedding Planner	a wedding consultant assists couples in planning and coordinating their weddings, helping them with vendor selection, budget management, and ensuring a seamless wedding day experience.	['Budget Management']

18. Dashboard & Data Visualization

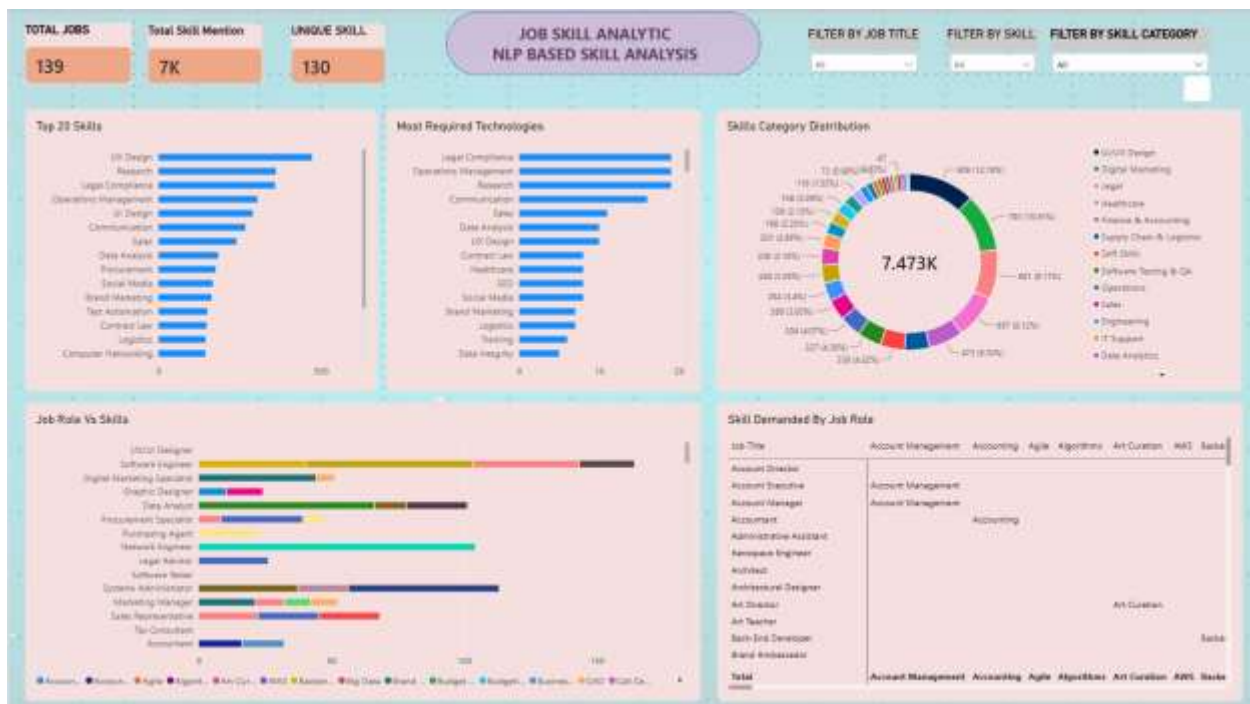
A Power BI dashboard (Job_Skill_Analytics.pbix) was built on an exploded export of the dataset — one row per (job, skill) pair, with the real taxonomy category attached to each row.

KPI Cards

- Total Jobs — distinct count of job postings.
- Unique Skills — distinct count of detected skills.
- Top Skill — most frequently mentioned skill overall.
- Top Job Role — job title with the most skill mentions.
- Top Skill Category — most frequently mentioned taxonomy category.
- Most Required Technology — top skill restricted to technical categories (Programming, Database, Cloud, etc.).

Charts

- Top 20 Skills — bar chart ranked by mention count.
- Skill Demand by Job Role — matrix of role × skill, filterable by job title.
- Skill Category Distribution — donut chart across the 37 taxonomy categories.
- Skill Trends — line chart of skill mentions by posting year (2021–2023).



19. Application / API Development

The extraction pipeline was packaged as a FastAPI backend with a companion static HTML/JS frontend, and a full account system (signup, login, sessions) was added so the extraction endpoint requires authentication.

Architecture

User → Web Interface → Authentication → Preprocessing → Skill Extraction Model (Taxonomy + spaCy union) → Skill Normalization → Category Mapping → Final Skills

Application Interface

welcome.html (landing page with Login/Sign Up), login.html and signup.html (account forms), and index.html (the extraction tool itself — a paste box and "Extract Skills" button, gated behind a valid login session).

Input / Output Example

Input: "Looking for Data Analyst with SQL, Python, Power BI and Excel experience."

Output: Python → Programming, SQL → Database, Power BI → Business Intelligence, Microsoft Excel → Excel & Office, plus a recommended technology (Power BI).

Authentication

User accounts are stored in SQLite with PBKDF2-hashed passwords (100,000 iterations, random per-user salt) and random session tokens stored as HttpOnly cookies — implemented entirely with Python's standard library to avoid unnecessary external dependencies for a project at this scale.

API Endpoints

Endpoint	Method	Purpose
/signup	POST	Create a new user account
/login	POST	Authenticate and set a session cookie
/logout	POST	Invalidate the current session
/me	GET	Return the currently logged-in user
/extract-skills	POST	Extract and categorize skills (requires login)
/health	GET	Basic service health check

20. Testing & Results

The full authentication and extraction flow was tested end-to-end against a live running server, not just reviewed as code:

- Signup with a new username/email/password — succeeded, account created.
- Login with those credentials — succeeded, session cookie issued.
- GET /me with the session cookie — correctly returned the logged-in user.
- POST /extract-skills with the session cookie — correctly returned categorized skills for a sample job description.
- POST /extract-skills without a session cookie — correctly rejected with HTTP 401 Unauthorized.
- Logout — session invalidated; a subsequent /me call with the same (now-expired) cookie correctly returned HTTP 401.

Sample Test Cases

Job Description	Skills Detected
Data Analyst — SQL, Python, Power BI, Excel	Python, SQL, Power BI, Microsoft Excel, Data Analysis
Backend Developer — PostgreSQL, AWS, CI/CD	PostgreSQL, AWS, API Development, DevOps
Digital Marketing Specialist — SEO, Google Analytics	SEO, Google Analytics, Content Marketing, Email Marketing
HR Manager — recruitment, onboarding, compliance	Recruitment, Onboarding, Legal Compliance

JOB SKILL EXTRACTOR

Paste a job description below to extract the skills it mentions.

"Looking for Data Analyst with SQL, Python, Power BI and Excel experience."

Extract Skills

Found 5 skills:

DATA ANALYTICS

Data Analysis

EXCEL & OFFICE

Microsoft Excel

BUSINESS INTELLIGENCE

Power BI

PROGRAMMING

Python

DATABASE

SQL

21. Challenges & Limitations

Problems Faced

- Dataset templating caused a serious, easy-to-miss train/test leakage bug in the ML classifier (Section 13).
- The pretrained transformer's label space made it fundamentally incapable of skill extraction out of the box, requiring a custom fine-tuning approach (Section 14).
- An evaluation-pipeline bug (mismatched random samples merged together) silently corrupted the Day-15 metrics before being traced and fixed (Section 16).
- Environment/versioning issues during deployment — Python 3.9 does not support the ``X | None`` type-union syntax used during initial development (which requires Python 3.10+), requiring a rewrite to ``Optional[X]``.
- Browser cookie behavior differs for pages opened directly from disk (`file://` origin) versus served over HTTP — the login system required the frontend to be served via a local HTTP server rather than opened directly, since session cookies are not reliably sent from a `file://` origin.

Model Limitations

- Recall plateaus at roughly 45–47% for the best-performing methods — a real ceiling caused by the taxonomy's finite synonym coverage, not a fixable software bug.
- The fine-tuned transformer, while a major improvement over the pretrained baseline, still underperforms the simpler dictionary-based methods and would benefit from more training epochs and more weakly-labeled examples.

Dataset Limitations

- Heavy description templating limits how much genuine language variation the ML and transformer models can learn from.
- No resume or candidate-side data exists in the dataset, limiting the project to job-posting-side extraction only (see Future Scope, Section 23).

22. Conclusion

This project took a real, messy, 5,000-row job postings dataset through the full lifecycle of an NLP system: exploration, cleaning, taxonomy design, four independently built and evaluated extraction methods, honest error analysis, and a deployed application with a companion analytics dashboard.

The most valuable outcome of the project was not any single metric, but the diagnostic process behind it: three of the project's headline numbers — a 100% classifier accuracy, a zero-extraction transformer, and a roughly 2% evaluation accuracy — were each traced back to a specific, fixable bug rather than accepted at face value. The final comparison between methods is trustworthy precisely because those issues were found and corrected before drawing conclusions.

The production system ships the Taxonomy + spaCy PhraseMatcher ensemble (F1 0.53–0.55), reached through a working, authenticated web application and a Power BI dashboard — with the fine-tuned transformer kept as an actively improving second track rather than shipped in its current, lower-scoring state.

23. Future Scope

- Resume-to-job matching: extract skills from a candidate's resume and compare against a target posting's requirements.
- Candidate ranking: score and rank multiple candidates against a single posting based on skill overlap.
- Skill-gap analysis: for a given candidate and target role, surface exactly which required skills are missing.
- Recommendation system: suggest learning resources or adjacent roles based on a candidate's current skill set.
- Wider taxonomy coverage: use the false-negative patterns found in error analysis to expand the synonym list and close real recall gaps.
- Further transformer fine-tuning: more training epochs, more weakly-labeled examples, and potentially a larger base model.
- Production hardening: authentication rate-limiting, monitoring, and a public deployment of the API.

24. Internship Learning Outcomes

Technical Skills

- End-to-end NLP pipeline design: preprocessing, rule-based matching, statistical and semantic techniques, custom NER, classical ML, and transformer fine-tuning.
- Practical experience with spaCy, scikit-learn, and Hugging Face Transformers, including weak-supervision techniques for generating training labels without manual annotation.
- Backend API development with FastAPI, including authentication (password hashing, session cookies) and SQLite persistence.
- Power BI dashboard design: DAX measures, KPI cards, and chart selection for business-facing analytics.

Analytical Skills

- Diagnosing root causes behind suspicious results rather than accepting them at face value — most notably the train/test leakage and evaluation-sample-mismatch bugs.
- Designing a fair, reproducible evaluation methodology (a single shared labeled sample, Precision/Recall/F1 over exact-match accuracy) for comparing fundamentally different extraction methods.

Project-Development Skills

- Structuring a long-running, multi-week project into clear, day-wise milestones with concrete deliverables at each stage.
- Producing documentation and reporting (day-wise write-ups, a model comparison report, and this final report) suitable for a non-technical audience as well as a technical one.

25. References

- Dataset: a 5,000-row job postings dataset with 23 fields, covering postings from 2021–2023 (title, description, qualifications, salary range, company details, and posting date).
- spaCy documentation — <https://spacy.io/>
- Hugging Face Transformers documentation — <https://huggingface.co/docs/transformers>
- Scikit-learn documentation — <https://scikit-learn.org/>
- FastAPI documentation — <https://fastapi.tiangolo.com/>
- Power BI documentation — <https://learn.microsoft.com/power-bi/>
- dslim/distilbert-NER pretrained model card — <https://huggingface.co/dslim/distilbert-NER>

26. Appendix

A. Sample API Request

POST /extract-skills

```
{ "job_description": "Looking for Data Analyst with SQL, Python, Power BI and Excel experience." }
```

B. Sample API Response

```
{ "skills": [{ "canonical_skill": "Python", "category": "Programming"}, { "canonical_skill": "SQL",  
"category": "Database"}, { "canonical_skill": "Power BI", "category": "Business Intelligence"},  
{ "canonical_skill": "Microsoft Excel", "category": "Excel & Office"}], "skill_count": 4,  
"recommended_technology": "Power BI" }
```

C. Core Extraction Function (simplified)

```
def extract_taxonomy_skills(text):  
    text = text.lower()  
    found = {}  
    for pattern, info in compiled_patterns:  
        if pattern.search(text):  
            found[info['canonical_skill']] = info['category']  
    return found
```

D. Deliverable File Structure

app.py,

model/taxonomy.pkl,

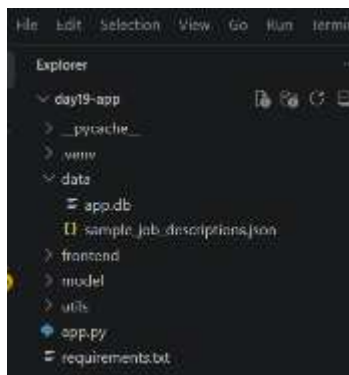
utils/extraction.py,

utils/auth.py,

data/sample_job_descriptions.json,

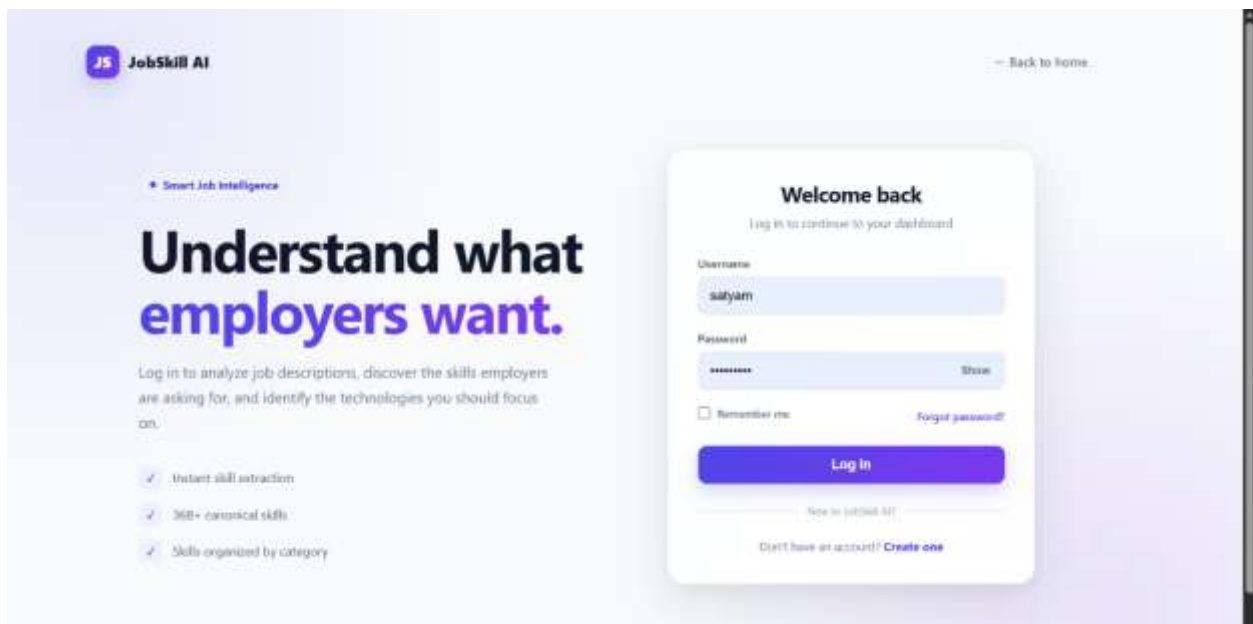
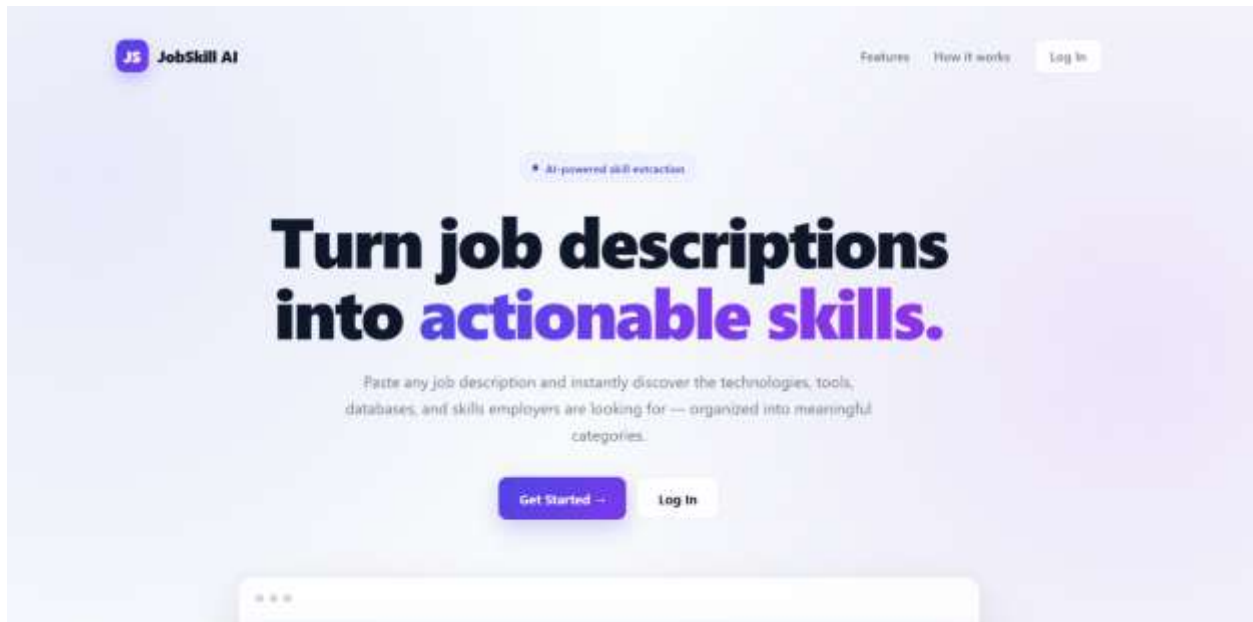
frontend/ (welcome.html, login.html, signup.html, index.html), requirements.txt,

README.md



E. Screenshots

Screenshots of the running application (welcome page, login, extraction results) and the Power BI dashboard to be inserted here from the live deployment.



LOGIN PAGE

JobSkill AI

Back to home

AI-Powered job intelligence

Start understanding job requirements.

Create your JobSkill AI account and turn complicated job descriptions into clear, actionable skill insights.

Instant skill extraction

Identify important technologies and skills from job descriptions in seconds.

Organized insights

Skills are automatically grouped into meaningful categories.

Built for job seekers

Understand what employers are actually looking for before you apply.

Create your account

Join JobSkill AI and start analyzing jobs.

Username

Choose a username

Email address

you@example.com

Password

Create a password

Show

Must be at least 8 characters with a mix of letters, numbers and symbols.

Create Account

Already have an account? Log in

SIGNUP PAGE

JobSkill AI

Hi, satyam Log Out

Home

AI Job Intelligence

Python

JavaScript

React

Java

Discover the skills employers want.

Paste a job description and instantly identify the technologies, tools and skills employers are looking for.

Job Description

0 characters

Paste the complete job description here...

Example

We are looking for a Data Analyst with strong SQL, Excel and Power BI experience. Knowledge of Python, Pandas and data visualization is preferred.

Tip: More detailed descriptions produce better results.

Extract Skills

Search

MAIN PAGE

Power BI

Extraction Results

Skills detected from the job description

Skills Detected

5

Categories

5

Recommended Technology

Power BI

Switch Source

Detected Skills

DATA ANALYTICS

Data Analysis

EXCEL & OFFICE

Microsoft Excel

BUSINESS INTELLIGENCE

Power BI

PROGRAMMING

Search

+

Recommended Technology

Based on the skills identified in this job posting, AISSUE is highlighting the most relevant technology.

Power BI

Search